```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.metrics import (
    accuracy_score, confusion_matrix, classification_report,
    ConfusionMatrixDisplay, roc_curve, roc_auc_score
)

sns.set_style("whitegrid")
plt.rcParams['figure.dpi'] = 100

RANDOM_STATE = 42
```

```
url = "https://raw.githubusercontent.com/sharmaroshan/Heart-UCI-Dataset/master/heart.csv"
df = pd.read_csv(url)

print("Shape:", df.shape)
df.head()
```

```
Shape: (303, 14)
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       303 non-null    int64
 1   sex       303 non-null    int64
 2   cp        303 non-null    int64
 3   trestbps  303 non-null    int64
 4   chol      303 non-null    int64
 5   fbs       303 non-null    int64
 6   restecg   303 non-null    int64
 7   thalach   303 non-null    int64
 8   exang     303 non-null    int64
 9   oldpeak   303 non-null    float64
 10  slope     303 non-null    int64
 11  ca        303 non-null    int64
 12  thal      303 non-null    int64
 13  target    303 non-null    int64
dtypes: float64(1), int64(13)
memory usage: 33.3 KB
```

```
df.isnull().sum()
```

	0
age	0
sex	0
cp	0
trestbps	0
chol	0
fbs	0
restecg	0
thalach	0
exang	0
oldpeak	0
slope	0
ca	0
thal	0
target	0

dtype: int64

```
df.describe().T
```

	count	mean	std	min	25%	50%	75%	max
age	303.0	54.366337	9.082101	29.0	47.5	55.0	61.0	77.0
sex	303.0	0.683168	0.466011	0.0	0.0	1.0	1.0	1.0
cp	303.0	0.966997	1.032052	0.0	0.0	1.0	2.0	3.0
trestbps	303.0	131.623762	17.538143	94.0	120.0	130.0	140.0	200.0
chol	303.0	246.264026	51.830751	126.0	211.0	240.0	274.5	564.0
fbs	303.0	0.148515	0.356198	0.0	0.0	0.0	0.0	1.0
restecg	303.0	0.528053	0.525860	0.0	0.0	1.0	1.0	2.0
thalach	303.0	149.646865	22.905161	71.0	133.5	153.0	166.0	202.0
exang	303.0	0.326733	0.469794	0.0	0.0	0.0	1.0	1.0
oldpeak	303.0	1.039604	1.161075	0.0	0.0	0.8	1.6	6.2
slope	303.0	1.399340	0.616226	0.0	1.0	1.0	2.0	2.0
ca	303.0	0.729373	1.022606	0.0	0.0	0.0	1.0	4.0
thal	303.0	2.313531	0.612277	0.0	2.0	2.0	3.0	3.0
target	303.0	0.544554	0.498835	0.0	0.0	1.0	1.0	1.0

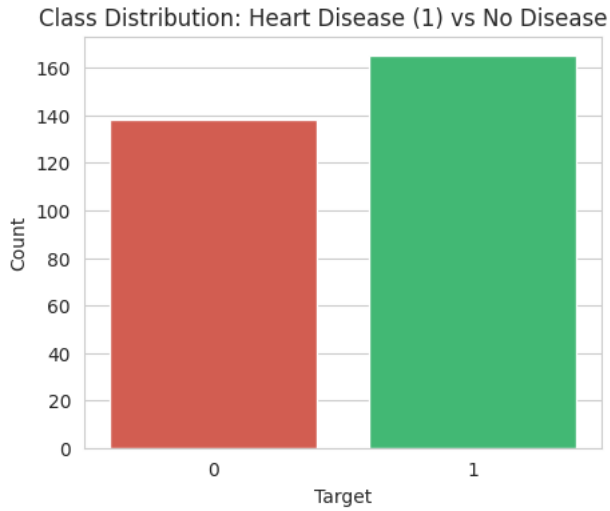
```
plt.figure(figsize=(5,4))
sns.countplot(x='target', data=df, palette=['#e74c3c', '#2ecc71'])
plt.title('Class Distribution: Heart Disease (1) vs No Disease (0)')
plt.xlabel('Target')
plt.ylabel('Count')
plt.show()

print(df['target'].value_counts(normalize=True).round(3) * 100, "% of total")
```

```
/tmp/ipykernel_2011/4148428831.py:2: FutureWarning:

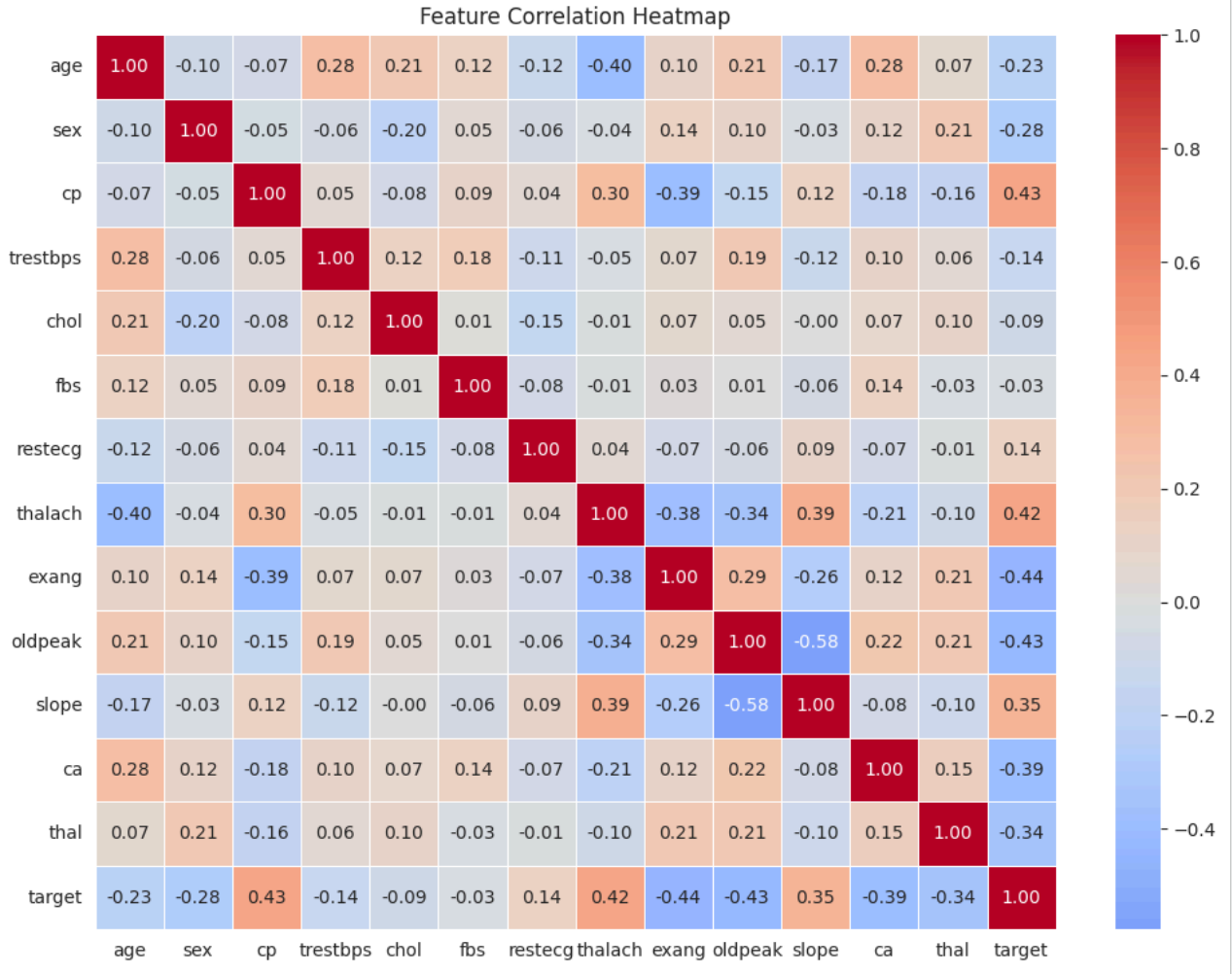
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and

  sns.countplot(x='target', data=df, palette=['#e74c3c', '#2ecc71'])
```




```
target
1    54.5
0    45.5
Name: proportion, dtype: float64 % of total
```

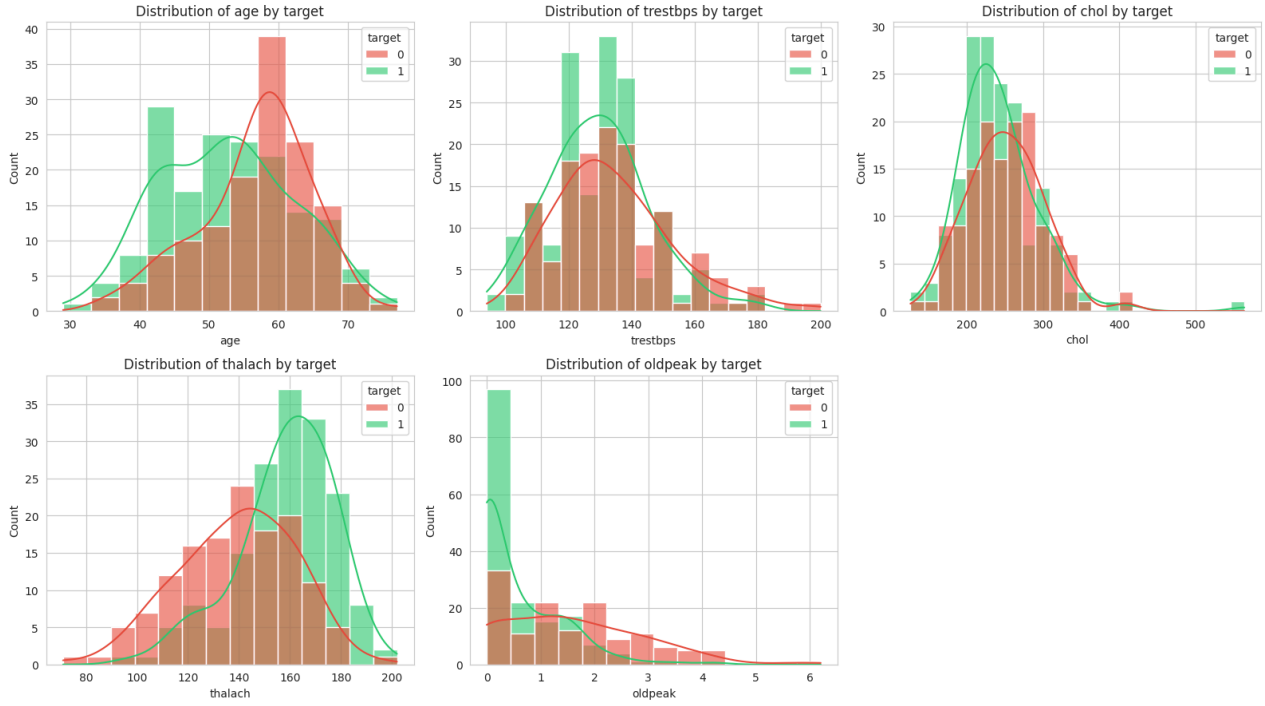
```
plt.figure(figsize=(12, 9))
corr = df.corr()
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', center=0, linewidths=0.5)
plt.title('Feature Correlation Heatmap')
plt.show()
```




```
fig, axes = plt.subplots(2, 3, figsize=(16, 9))
num_features = ['age', 'trestbps', 'chol', 'thalach', 'oldpeak']
```

```
for ax, col in zip(axes.flatten(), num_features):
    sns.histplot(data=df, x=col, hue='target', kde=True, ax=ax, palette=['#e74c3c', '#2ecc71'], alpha=0.6)
    ax.set_title(f'Distribution of {col} by target')

axes[1, 2].axis('off')
plt.tight_layout()
plt.show()
```




```
X = df.drop('target', axis=1)
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
)

print("Train shape:", X_train.shape)
print("Test shape :", X_test.shape)
```

```
Train shape: (242, 13)
Test shape : (61, 13)
```

```
dt_gini = DecisionTreeClassifier(
    criterion='gini',
    max_depth=4,
    min_samples_split=10,
    min_samples_leaf=5,
    random_state=RANDOM_STATE
)

dt_gini.fit(X_train, y_train)
```

```
DecisionTreeClassifier(max_depth=4, min_samples_leaf=5, min_samples_split=10,
                       random_state=42)
```

```
y_pred = dt_gini.predict(X_test)
y_prob = dt_gini.predict_proba(X_test)[:, 1]

acc = accuracy_score(y_test, y_pred)
print(f"Test Accuracy: {acc:.4f}\n")
print("Classification Report:\n")
print(classification_report(y_test, y_pred, target_names=['No Disease', 'Disease']))
```

```
Test Accuracy: 0.7869

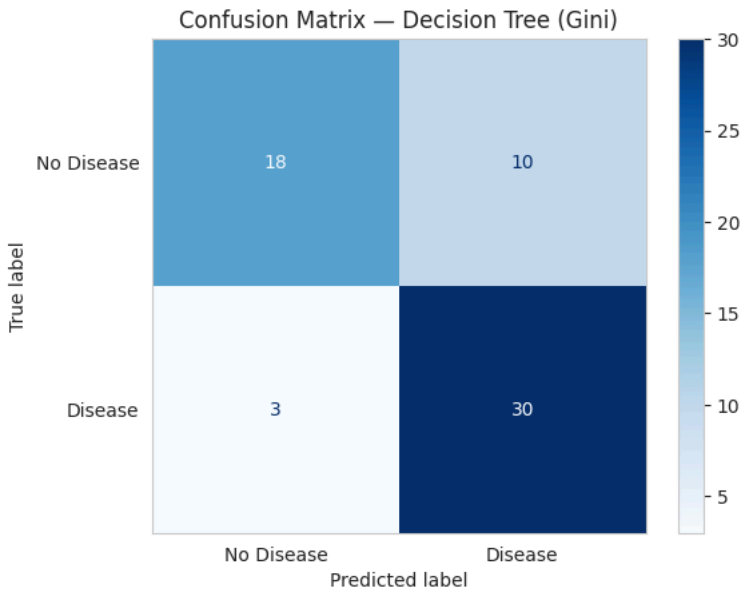
Classification Report:

              precision    recall  f1-score   support

  No Disease       0.86      0.64      0.73        28
     Disease       0.75      0.91      0.82        33

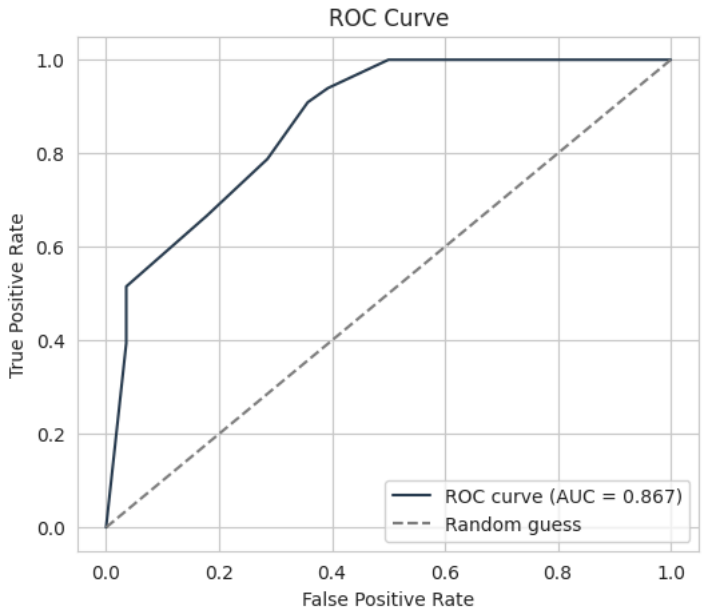
    accuracy                           0.79        61
   macro avg       0.80      0.78      0.78        61
weighted avg       0.80      0.79      0.78        61
```

```
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['No Disease', 'Disease'])
disp.plot(cmap='Blues', values_format='d')
plt.title('Confusion Matrix — Decision Tree (Gini)')
plt.grid(False)
plt.show()
```

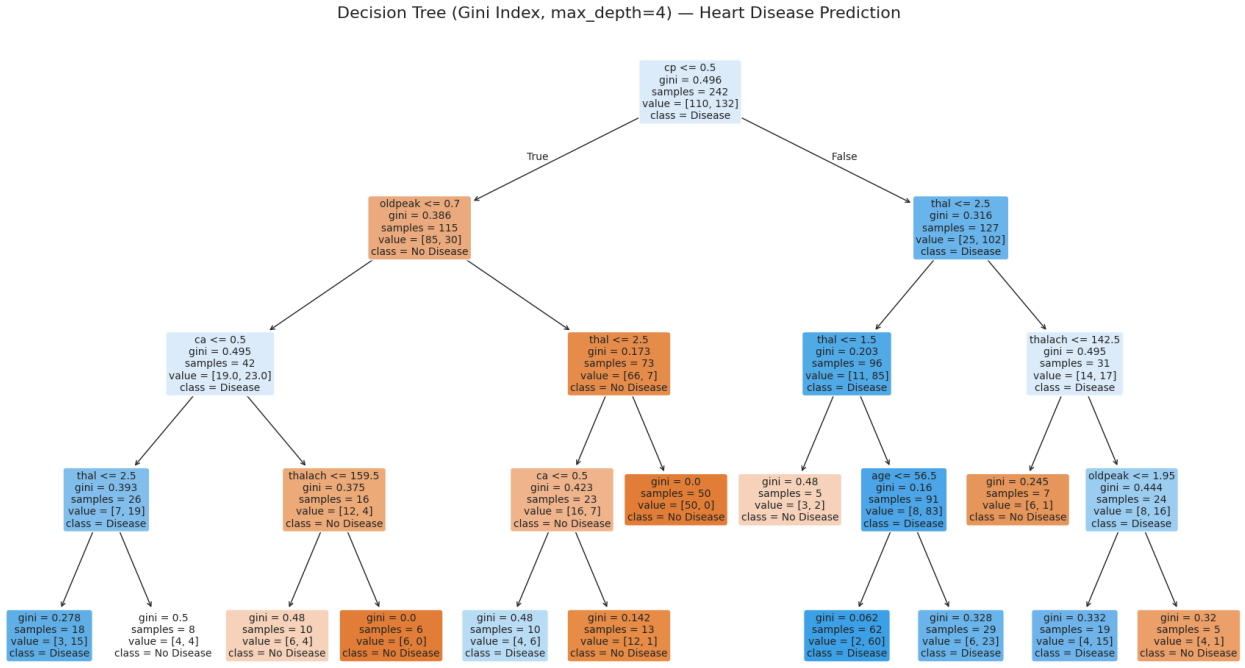



```
fpr, tpr, _ = roc_curve(y_test, y_prob)
auc_score = roc_auc_score(y_test, y_prob)

plt.figure(figsize=(6, 5))
plt.plot(fpr, tpr, label=f'ROC curve (AUC = {auc_score:.3f})', color='#2c3e50')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Random guess')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend()
plt.show()
```




```
plt.figure(figsize=(22, 12))
plot_tree(
    dt_gini,
    feature_names=X.columns,
    class_names=['No Disease', 'Disease'],
    filled=True,
    rounded=True,
    fontsize=10,
    proportion=False,
    impurity=True
)
plt.title("Decision Tree (Gini Index, max_depth=4) — Heart Disease Prediction", fontsize=16)
plt.show()
```



```
rules_text = export_text(dt_gini, feature_names=list(X.columns))
print(rules_text)
```

```
|--- cp <= 0.50
|   |--- oldpeak <= 0.70
|   |   |--- ca <= 0.50
|   |   |   |--- thal <= 2.50
|   |   |   |   |--- class: 1
|   |   |   |--- thal >  2.50
|   |   |   |   |--- class: 0
|   |   |--- ca >  0.50
|   |   |   |--- thalach <= 159.50
|   |   |   |   |--- class: 0
|   |   |   |--- thalach >  159.50
|   |   |   |   |--- class: 0
|   |--- oldpeak >  0.70
|   |   |--- thal <= 2.50
|   |   |   |--- ca <= 0.50
|   |   |   |   |--- class: 1
|   |   |   |--- ca >  0.50
|   |   |   |   |--- class: 0
|   |   |--- thal >  2.50
|   |   |   |--- class: 0
|--- cp >  0.50
|   |--- thal <= 2.50
|   |   |--- thal <= 1.50
|   |   |   |--- class: 0
|   |   |--- thal >  1.50
|   |   |   |--- age <= 56.50
|   |   |   |   |--- class: 1
|   |   |   |--- age >  56.50
|   |   |   |   |--- class: 1
|   |--- thal >  2.50
|   |   |--- thalach <= 142.50
|   |   |   |--- class: 0
|   |   |--- thalach >  142.50
|   |   |   |--- oldpeak <= 1.95
|   |   |   |   |--- class: 1
|   |   |   |--- oldpeak >  1.95
|   |   |   |   |--- class: 0
```

```
def extract_rules(tree, feature_names, class_names):
    tree_ = tree.tree_
    feature_name = [
        feature_names[i] if i != -2 else "undefined!"
        for i in tree_.feature
    ]
    rules = []

    def recurse(node, path):
        if tree_.feature[node] != -2:  # not a leaf
            name = feature_name[node]
            threshold = tree_.threshold[node]

            left_path = path + [f"{name} <= {threshold:.2f}"]
            recurse(tree_.children_left[node], left_path)

            right_path = path + [f"{name} > {threshold:.2f}"]
            recurse(tree_.children_right[node], right_path)
        else:  # leaf node
            values = tree_.value[node][0]
            predicted_class = class_names[int(np.argmax(values))]
            n_samples = int(values.sum())
            confidence = values.max() / values.sum() * 100
            rule = "IF " + " AND ".join(path) + \
                   f"  =>  THEN predict '{predicted_class}'" \
                   f"  (confidence: {confidence:.1f}%, samples: {n_samples})"
            rules.append(rule)

    recurse(0, [])
    return rules


rules = extract_rules(dt_gini, list(X.columns), ['No Disease', 'Disease'])

print(f"Total decision rules extracted: {len(rules)}\n")
for i, r in enumerate(rules, 1):
    print(f"Rule {i}: {r}\n")
```

```
Total decision rules extracted: 13

Rule 1: IF cp <= 0.50 AND oldpeak <= 0.70 AND ca <= 0.50 AND thal <= 2.50  =>  THEN predict 'Disease'  (confidence: 83.3%, s
```

```
Rule 2: IF cp <= 0.50 AND oldpeak <= 0.70 AND ca <= 0.50 AND thal > 2.50  =>  THEN predict 'No Disease'  (confidence: 50.0%,

Rule 3: IF cp <= 0.50 AND oldpeak <= 0.70 AND ca > 0.50 AND thalach <= 159.50  =>  THEN predict 'No Disease'  (confidence: 6

Rule 4: IF cp <= 0.50 AND oldpeak <= 0.70 AND ca > 0.50 AND thalach > 159.50  =>  THEN predict 'No Disease'  (confidence: 10

Rule 5: IF cp <= 0.50 AND oldpeak > 0.70 AND thal <= 2.50 AND ca <= 0.50  =>  THEN predict 'Disease'  (confidence: 60.0%, sa

Rule 6: IF cp <= 0.50 AND oldpeak > 0.70 AND thal <= 2.50 AND ca > 0.50  =>  THEN predict 'No Disease'  (confidence: 92.3%,

Rule 7: IF cp <= 0.50 AND oldpeak > 0.70 AND thal > 2.50  =>  THEN predict 'No Disease'  (confidence: 100.0%, samples: 1)

Rule 8: IF cp > 0.50 AND thal <= 2.50 AND thal <= 1.50  =>  THEN predict 'No Disease'  (confidence: 60.0%, samples: 1)

Rule 9: IF cp > 0.50 AND thal <= 2.50 AND thal > 1.50 AND age <= 56.50  =>  THEN predict 'Disease'  (confidence: 96.8%, samp

Rule 10: IF cp > 0.50 AND thal <= 2.50 AND thal > 1.50 AND age > 56.50  =>  THEN predict 'Disease'  (confidence: 79.3%, samp

Rule 11: IF cp > 0.50 AND thal > 2.50 AND thalach <= 142.50  =>  THEN predict 'No Disease'  (confidence: 85.7%, samples: 1)

Rule 12: IF cp > 0.50 AND thal > 2.50 AND thalach > 142.50 AND oldpeak <= 1.95  =>  THEN predict 'Disease'  (confidence: 78.

Rule 13: IF cp > 0.50 AND thal > 2.50 AND thalach > 142.50 AND oldpeak > 1.95  =>  THEN predict 'No Disease'  (confidence: 8
```

```
importance_df = pd.DataFrame({
    'Feature': X.columns,
    'Importance': dt_gini.feature_importances_
}).sort_values('Importance', ascending=False)

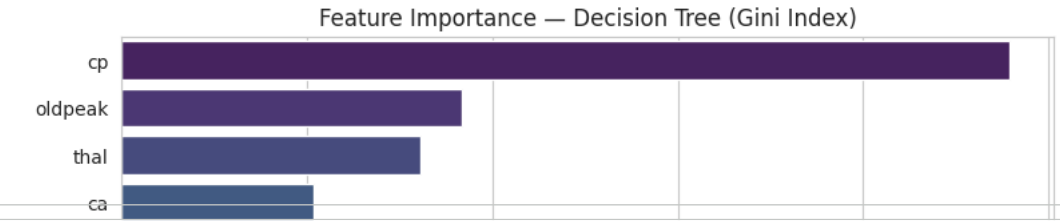
plt.figure(figsize=(9, 6))
sns.barplot(data=importance_df, x='Importance', y='Feature', palette='viridis')
plt.title('Feature Importance — Decision Tree (Gini Index)')
plt.xlabel('Gini Importance')
plt.show()

importance_df
```

```
/tmp/ipykernel_2011/1721095308.py:7: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` a

  sns.barplot(data=importance_df, x='Importance', y='Feature', palette='viridis')
```




```
depths = range(1, 15)
train_scores, test_scores = [], []

for d in depths:
    clf = DecisionTreeClassifier(criterion='gini', max_depth=d, random_state=RANDOM_STATE)
    clf.fit(X_train, y_train)
    train_scores.append(clf.score(X_train, y_train))
    test_scores.append(clf.score(X_test, y_test))

plt.figure(figsize=(8, 5))
plt.plot(depths, train_scores, marker='o', label='Train Accuracy')
plt.plot(depths, test_scores, marker='s', label='Test Accuracy')
plt.xlabel('max_depth')
plt.ylabel('Accuracy')
plt.title('Train vs Test Accuracy across Tree Depths')
plt.legend()
plt.grid(True)
plt.show()

best_depth = depths[int(np.argmax(test_scores))]
print(f"Best max_depth on test set: {best_depth} (Test Accuracy: {max(test_scores):.4f})")
```

